

Chapter 15

Condition Number Estimation

*Most of LAPACK's condition numbers and error bounds are based on
estimated condition numbers ...
The price one pays for using an estimated
rather than an exact condition number is
occasional (but very rare) underestimates of the true error;
years of experience attest to the reliability of our estimators,
although examples where they badly underestimate the error can be constructed.*
— E. ANDERSON et al., *LAPACK Users' Guide* (1999)

*The importance of the counter-examples is that they make clear that
any effort toward proving that the algorithms
always produce useful estimations is fruitless.
It may be possible to prove that the algorithms
produce useful estimations in certain situations, however,
and this should be pursued.
An effort simply to construct more complex algorithms is dangerous.*
— A. K. CLINE and R. K. REW, *A Set of Counter-Examples
to Three Condition Number Estimators* (1983)

Singularity is almost invariably a clue.
— SIR ARTHUR CONAN DOYLE, *The Boscombe Valley Mystery* (1892)

Condition number estimation is the process of computing an inexpensive estimate of a condition number, where “inexpensive” usually means that the cost is an order of magnitude less than would be required to compute the condition number exactly. Condition estimates are required when evaluating forward error bounds for computed solutions to many types of linear algebra problems. An estimate of the condition number that is correct to within a factor 10 is usually acceptable, because it is the magnitude of an error bound that is of interest, not its precise value. In this chapter we are concerned with condition numbers involving A^{-1} , where A is a given nonsingular $n \times n$ matrix; these arise in linear systems $Ax = b$ but also more generally in, for example, least squares problems and eigenvalue problems.

An important question for any condition estimator is whether there exists a “counterexample”: a parametrized matrix for which the quotient “estimate divided by true condition number” can be made arbitrarily small (or large, depending on whether the estimate is a lower bound or an upper bound) by varying a parameter. Intuitively, it might be expected that any condition estimator has counterexamples or, equivalently, that estimating a condition number to within a constant factor independent of the matrix A must cost as much, asymptotically, as computing A^{-1} . This property was conjectured by Demmel [314, 1992] and strong evidence for it is provided by Demmel, Diament, and Malajovich [321, 2001]. In the latter paper it is shown that the cost of computing an estimate of $\|A^{-1}\|$ of guaranteed quality is at least the cost of testing whether the product of two $n \times n$ matrices is zero, and performing this test is conjectured to cost as much as actually computing the product [185, 1997, Problem 16.3].

15.1. How to Estimate Componentwise Condition Numbers

In the perturbation theory of Chapter 7 for linear equations we obtained perturbation bounds that involve the condition numbers

$$\kappa_{E,f}(A, x) = \frac{\|A^{-1}\| \|f\|}{\|x\|} + \|A^{-1}\| \|E\|,$$

$$\text{cond}_{E,f}(A, x) = \frac{\| |A^{-1}| (|E|x + f) \|_{\infty}}{\|x\|_{\infty}},$$

and their variants. To compute these condition numbers exactly we need to compute A^{-1} , which requires $O(n^3)$ operations, even if we are given a factorization of A . Is it possible to produce reliable estimates of both condition numbers in $O(n^2)$ operations? The answer is yes, but to see why we first need to rewrite the expression for $\text{cond}_{E,f}(A, x)$. Consider the general expression $\| |A^{-1}| d \|_{\infty}$, where d is a given nonnegative vector (thus, $d = f + E|x|$ for $\text{cond}_{E,f}(A, x)$); note that the practical error bound (7.31) is of this form. Writing $D = \text{diag}(d)$ and $e = [1, 1, \dots, 1]^T$, we have

$$\| |A^{-1}| d \|_{\infty} = \| |A^{-1}| D e \|_{\infty} = \| |A^{-1}| D \| e \|_{\infty} = \| |A^{-1}| D \|_{\infty} = \| A^{-1} D \|_{\infty}. \quad (15.1)$$

Hence the problem is equivalent to that of estimating $\|B\|_\infty := \|A^{-1}D\|_\infty$. If we can estimate this quantity then we can estimate any of the condition numbers or perturbation bounds for a linear system. There is an algorithm, described in §15.3, that produces reliable order-of-magnitude estimates of $\|B\|_1$, for an arbitrary B , by computing just a few matrix–vector products Bx and B^Ty for carefully selected vectors x and y and then approximating $\|B\|_1 \approx \|Bx\|_1/\|x\|_1$. If we assume that we have a factorization of A (say, $PA = LU$ or $A = QR$), then we can form the required matrix–vector products for $B = A^{-1}D$ in $O(n^2)$ flops. Since $\|B\|_1 = \|B^T\|_\infty$, it follows that we can use the algorithm to estimate $\|A^{-1}d\|_\infty$ in $O(n^2)$ flops.

Before presenting the 1-norm condition estimation algorithm, we describe a more general method that estimates $\|B\|_p$ for any $p \in [1, \infty]$. This more general method is of interest in its own right and provides insight into the special case $p = 1$.

15.2. The p -Norm Power Method

An iterative “power method” for computing $\|A\|_p$ was proposed by Boyd in 1974 [155, 1974]. When $p = 2$ it reduces to the usual power method applied to A^TA . We use the notation $\text{dual}_p(x)$ to denote any vector y of unit q -norm such that equality holds in the Hölder inequality $x^Ty \leq \|x\|_p\|y\|_q$ (this normalization is different from the one we used in (6.3), but is more convenient for our present purposes). Throughout this section, $p \geq 1$ and q is defined by $p^{-1} + q^{-1} = 1$.

Algorithm 15.1 (p -norm power method). Given $A \in \mathbb{R}^{m \times n}$ and $x_0 \in \mathbb{R}^n$, this algorithm computes γ and x such that $\gamma \leq \|A\|_p$ and $\|Ax\|_p = \gamma\|x\|_p$.

```

 $x = x_0/\|x_0\|_p$ 
repeat
   $y = Ax$ 
   $z = A^T \text{dual}_p(y)$ 
  if  $\|z\|_q \leq z^Tx$ 
     $\gamma = \|y\|_p$ 
    quit
  end
   $x = \text{dual}_q(z)$ 
end
```

Cost: $4rmn$ flops (for r iterations).

There are several ways to derive Algorithm 15.1. Perhaps the most natural way is to examine the optimality conditions for maximization of

$$F(x) = \frac{\|Ax\|_p}{\|x\|_p}, \quad x \in \mathbb{R}^n.$$

First, we note that the subdifferential (that is, the set of subgradients) of an arbitrary vector norm $\|\cdot\|$ is given by [417, 1987, p. 379]

$$\partial\|x\| = \{ \lambda : \lambda^Tx = \|x\|, \|\lambda\|_D \leq 1 \}.$$

If $x \neq 0$ then $\lambda^T x = \|x\| \Rightarrow \|\lambda\|_D \geq 1$, from the Hölder inequality, and so, if $x \neq 0$,

$$\begin{aligned}\partial\|x\| &= \{ \lambda : \lambda^T x = \|x\|, \|\lambda\|_D = 1 \} \\ &=: \{\text{dual}(x)\}.\end{aligned}$$

It can also be shown that if A has full rank,

$$\partial\|Ax\| = \{A^T \text{dual}(Ax)\}. \quad (15.2)$$

We assume now that A has full rank, $1 < p < \infty$, and $x \neq 0$. Then it is easy to see that there is a unique vector $\text{dual}_p(x)$, so $\partial\|x\|_p$ has just one element, that is, $\|x\|_p$ is differentiable. Hence we have

$$\nabla F(x) = \frac{A^T \text{dual}_p(Ax)}{\|x\|_p} - \frac{\|Ax\|_p \text{dual}_p(x)}{\|x\|_p^2}. \quad (15.3)$$

The first-order Kuhn–Tucker condition for a local maximum of F is therefore

$$A^T \text{dual}_p(Ax) = \frac{\|Ax\|_p}{\|x\|_p} \text{dual}_p(x).$$

Since $\text{dual}_q(\text{dual}_p(x)) = x/\|x\|_p$ if $p \neq 1, \infty$, this equation can be written

$$x = \frac{\|x\|_p^2}{\|Ax\|_p} \text{dual}_q(A^T \text{dual}_p(Ax)). \quad (15.4)$$

The power method is simply functional iteration applied to this transformed set of Kuhn–Tucker equations (the scale factor $\|x\|_p^2/\|Ax\|_p$ is irrelevant since $F(\alpha x) = F(x)$).

For the 1- and ∞ -norms, which are not everywhere differentiable, a different derivation can be given. The problem can be phrased as one of maximizing the convex function $F(x) := \|Ax\|_p$ over the convex set $S := \{x : \|x\|_p \leq 1\}$. The convexity of F and S ensures that, for any $u \in S$, at least one vector g exists such that

$$F(v) \geq F(u) + g^T(v - u) \quad \text{for all } v \in S. \quad (15.5)$$

Vectors g satisfying (15.5) are called *subgradients* of F (see, for example, [417, 1987, p. 364]). Inequality (15.5) suggests the strategy of choosing a subgradient g and moving from u to a point $v_* \in S$ that maximizes $g^T(v - u)$, that is, a vector that maximizes $g^T v$. Clearly, $v_* = \text{dual}_q(g)$. Since, from (15.2), g has the form $A^T \text{dual}_p(Ax)$, this completes the derivation of the iteration.

For all values of p the power method has the desirable property of generating an increasing sequence of norm approximations.

Lemma 15.2. *In Algorithm 15.1, the vectors from the k th iteration satisfy*

- (a) $z^{k^T} x^k = \|y^k\|_p$, and
- (b) $\|y^k\|_p \leq \|z^k\|_q \leq \|y^{k+1}\|_p \leq \|A\|_p$.

The first inequality in (b) is strict if convergence is not obtained on the k th iteration.

Proof. $z^{k^T} x^k = \text{dual}_p(y^k)^T A x^k = \text{dual}_p(y^k)^T y^k = \|y^k\|_p$. Then

$$\begin{aligned} \|y^k\|_p &= z^{k^T} x^k \\ &\leq \|z^k\|_q \|x^k\|_p = \|z^k\|_q = z^{k^T} x^{k+1} = \text{dual}_p(y^k)^T A x^{k+1} \\ &\leq \|\text{dual}_p(y^k)\|_q \|A x^{k+1}\|_p = \|y^{k+1}\|_p \\ &\leq \|A\|_p. \end{aligned}$$

For the last part, note that, in view of (a), the convergence test “ $\|z^k\|_q \leq z^{k^T} x^k$ ” can be written as “ $\|z^k\|_q \leq \|y^k\|_p$ ”. $\square$

It is clear from Lemma 15.2 (or directly from the Hölder inequality) that the convergence test “ $\|z\|_q \leq z^T x$ ” in Algorithm 15.1 is equivalent to “ $\|z\|_q = z^T x$ ” and, since $\|x\|_p = 1$, this is equivalent to $x = \text{dual}_q(z)$. Thus, although the convergence test compares two scalars, it is actually testing for equality in the vector equation (15.4).

The convergence properties of Algorithm 15.1 merit a careful description. First, in view of Lemma 15.2, the scalars $\gamma_k = \|y^k\|_p$ form an increasing and convergent sequence. This does not necessarily imply that Algorithm 15.1 converges, since the algorithm tests for convergence of the x^k , and these vectors could fail to converge. However, a subsequence of the x^k must converge to a limit, $\bar{x}$ say. Boyd [155, 1974] shows that if $\bar{x}$ is a strong local maximum of F with no zero components, then $x^k \rightarrow \bar{x}$ linearly.

If Algorithm 15.1 converges, it converges to a stationary point of $F(x)$ when $1 < p < \infty$. Thus, instead of the desired global maximum $\|A\|_p$, we may obtain only a local maximum or even a saddle point. When $p = 1$ or ∞ , if the algorithm converges to a point at which F is not differentiable, that point need not even be a stationary point. On the other hand, for $p = 1$ or ∞ Algorithm 15.1 terminates in at most $n + 1$ iterations (assuming that when dual_p or dual_q is not unique an extreme point of the unit ball is taken), since the algorithm moves between the vertices e_i of the unit ball in the 1-norm, increasing F on each stage ($x = \pm e_i$ for $p = 1$, and $\text{dual}_p(y) = \pm e_i$ for $p = \infty$). An example where n iterations are required for $p = 1$ is given in Problem 15.3.

For two special types of matrix, more can be said about Algorithm 15.1.

(1) If $A = xy^T$ (rank 1), the algorithm converges on the second step with $\gamma = \|A\|_p = \|x\|_p \|y\|_q$, whatever x_0 .

(2) Boyd [155, 1974] shows that if A has nonnegative elements, $A^T A$ is irreducible, $1 < p < \infty$, and x_0 has positive elements, then the x^k converge and $\gamma_k \rightarrow \|A\|_p$.

For values of p strictly between 1 and ∞ the convergence behaviour of Algorithm 15.1 is typical of a linearly convergent method: exact convergence is not usually obtained in a finite number of steps and arbitrarily many steps can be required for convergence, as is well known for the 2-norm power method. Fortunately, there is a method for choosing a good starting vector that can be combined with Algorithm 15.1 to produce a reliable norm estimator; see the Notes and References and Problem 15.2.

We now turn our attention to the extreme values of p : 1 and ∞ .

15.3. LAPACK 1-Norm Estimator

Algorithm 15.1 has two remarkable properties when $p = 1$: it almost always converges within four iterations (when $x_0 = [1, 1, \dots, 1]^T$, say) and it frequently yields $\|A\|_1$ exactly. This rapid, finite termination is also obtained for $p = \infty$ and is related to the fact that Algorithm 15.1 moves among the finite set of extreme points of the unit ball. Numerical experiments suggest that the accuracy is about the same for both norms but that slightly more iterations are required on average for $p = \infty$. Hence we will confine our attention to the 1-norm.

The 1-norm version of Algorithm 15.1 was derived independently of the general algorithm by Hager [536, 1984] and can be expressed as follows. The notation $\xi = \text{sign}(y)$ means that $\xi_i = 1$ or -1 according as $y_i \geq 0$ or $y_i < 0$. We now specialize to square matrices.

Algorithm 15.3 (1-norm power method). Given $A \in \mathbb{R}^{n \times n}$ this algorithm computes γ and x such that $\gamma \leq \|A\|_1$ and $\|Ax\|_1 = \gamma\|x\|_1$.

```

 $x = n^{-1}e$ 
repeat
   $y = Ax$ 
   $\xi = \text{sign}(y)$ 
   $z = A^T \xi$ 
  if  $\|z\|_\infty \leq z^T x$ 
     $\gamma = \|y\|_1$ 
    quit
  end
   $x = e_j$ , where  $|z_j| = \|z\|_\infty$  (smallest such  $j$ )
end

```

Numerical experiments show that the estimates produced by Algorithm 15.3 are frequently exact ($\gamma = \|A\|_1$), usually “acceptable” ($\gamma \geq \|A\|_1/10$), and sometimes poor ($\gamma < \|A\|_1/10$).

A general class of counterexamples for Algorithm 15.3 is given by the matrices

$$A = I + \theta C,$$

where $Ce = C^T e = 0$ (there are many possible choices for C). For any such matrix, Algorithm 15.3 computes $y = n^{-1}e$, $\xi = e$, $z = e$, and hence the algorithm terminates at the end of the first iteration with

$$\frac{\gamma}{\|A\|_1} = \frac{1}{\|I + \theta C\|_1} \sim \theta^{-1} \quad \text{as } \theta \rightarrow \infty.$$

The problem is that the algorithm stops at a local maximum that can differ from the global one by an arbitrarily large factor.

A more reliable and more robust algorithm is produced by the following modifications of Higham [585, 1988].

Definition of estimate. To overcome most of the poor estimates, γ is redefined as

$$\gamma = \max \left\{ \|y\|_1, \frac{\|c\|_1}{\|b\|_1} \right\},$$

where

$$c = Ab, \quad b_i = (-1)^{i+1} \left(1 + \frac{i-1}{n-1} \right).$$

The vector b is considered likely to “pick out” any large elements of A in those cases where such elements fail to propagate through to y .

Convergence test. The algorithm is limited to a minimum of two and a maximum of five iterations. Further, convergence is declared after computing ξ if the new ξ is parallel to the previous one¹²; this event signals that convergence will be obtained on the current iteration and that the next (and final) multiplication $A^T \xi$ is unnecessary. Convergence is also declared if the new $\|y\|_1$ is no larger than the previous one. This nonincrease of the norm can happen only in finite precision arithmetic and signals the possibility of a vertex e_j being revisited—the onset of “cycling”.

The improved algorithm is as follows. This algorithm is the basis of all the condition number estimation in LAPACK.

Algorithm 15.4 (LAPACK norm estimator). Given $A \in \mathbb{R}^{n \times n}$ this algorithm computes γ and $v = Aw$ such that $\gamma \leq \|A\|_1$ with $\|v\|_1/\|w\|_1 = \gamma$ (w is not returned).

```

 $v = A(n^{-1}e)$ 
if  $n = 1$ , quit with  $\gamma = |v_1|$ , end
 $\gamma = \|v\|_1$ 
 $\xi = \text{sign}(v)$ 
 $x = A^T \xi$ 
 $k = 2$ 
repeat
     $j = \min\{i: |x_i| = \|x\|_\infty\}$ 
     $v = Ae_j$ 
     $\bar{\gamma} = \gamma$ 
     $\gamma = \|v\|_1$ 
    if  $\text{sign}(v) = \pm \xi$  or  $\gamma \leq \bar{\gamma}$ , goto (*), end
     $\xi = \text{sign}(v)$ 
     $x = A^T \xi$ 
     $k = k + 1$ 
until ( $\|x\|_\infty = x_j$  or  $k > 5$ )
(*)  $x_i = (-1)^{i+1} \left( 1 + \frac{i-1}{n-1} \right)$ ,  $i = 1:n$ 
 $x = Ax$ 
if  $2\|x\|_1/(3n) > \gamma$  then
     $v = x$ 
     $\gamma = 2\|x\|_1/(3n)$ 
end

```

Algorithm 15.4 can still be “defeated”: it returns an estimate 1 for matrices $A(\theta)$ of the form

$$A(\theta) = I + \theta P, \quad \text{where } P = P^T, \quad Pe = 0, \quad Pe_1 = 0, \quad Pb = 0. \quad (15.6)$$

¹²In [585, 1988] and in the algorithm used in LAPACK, only *equality* of the new and old ξ is tested, due to an oversight.

(P can be constructed as $I - Q$, where Q is the orthogonal projection onto $\text{span}\{e, e_1, b\}$.) Indeed, the existence of counterexamples is intuitively obvious since Algorithm 15.4 samples the behaviour of A on fewer than n vectors in $\mathbb{R}^n$. Numerical counterexamples (not parametrized) can be constructed automatically by direct search, as described in §26.3.1. Despite these weaknesses, practical experience with Algorithm 15.4 shows that it is very rare for the estimate to be more than three times smaller than the actual norm, independent of the dimension n . Therefore Algorithm 15.4 is, in practice, a *very reliable* norm estimator. The number of matrix–vector products required is at least 4 and at most 11, and averages between 4 and 5.

There is an analogue of Algorithm 15.3 for complex matrices, in which ξ_i is defined as $y_i/|y_i|$ if $y_i \neq 0$ and 1 otherwise. In the corresponding version of Algorithm 15.4 the test for parallel consecutive ξ vectors is removed, because ξ now has noninteger, complex components and so is unlikely to “repeat”.

MATLAB has a built-in function `rcond` that implements Algorithm 15.4.

It is interesting to look at a performance profile of Algorithm 15.4. A performance profile is a plot of some measure of the performance of an algorithm versus a problem parameter (the idea originates in quadrature—see [800, 1976]). In this case, the natural measure of performance is the underestimation ratio, $\gamma/\|A\|_1$. Figure 15.1 shows the performance profile for a 5×5 matrix $A(\theta)$ of the form (15.6), with P constructed as described above (because of rounding errors in constructing $A(\theta)$ and within the algorithm, the computed norm estimates differ from those that would be produced in exact arithmetic). The jagged nature of the performance curve is typical for algorithms that contain logical tests and branches. Small changes in the parameter θ , which themselves result in different rounding errors, can cause the algorithm to visit different vertices in this example.

15.4. Block 1-Norm Estimator

Algorithm 15.4 has two limitations. First, it offers the user no way to control or improve the accuracy of the estimate. Therefore it is not well suited to applications in which an estimate with one or more correct digits is required. Second, it is based on level-2 BLAS operations, which implies less than optimal efficiency on modern computers.

Higham and Tisseur [624, 2000] derive a block version of the 1-norm power method that iterates with an $n \times t$ matrix, where $t \geq 1$ is a parameter; for $t = 1$, Algorithm 15.3 is recovered. They develop this method into a practical algorithm generalizing Algorithm 15.4. The algorithm has several key features.

- The accuracy and reliability of the estimates it produces generally increase with t , leading quickly to estimates with one or more correct digits.
- The number of iterations varies little with t , so the fast convergence for $t = 1$ is maintained.
- The computational kernels are level-3 BLAS operations for $t > 1$.
- The first column of the starting matrix is $n^{-1}e$ (as in Algorithm 15.4) and the last $t - 1$ columns are chosen randomly, giving the algorithm a statistical

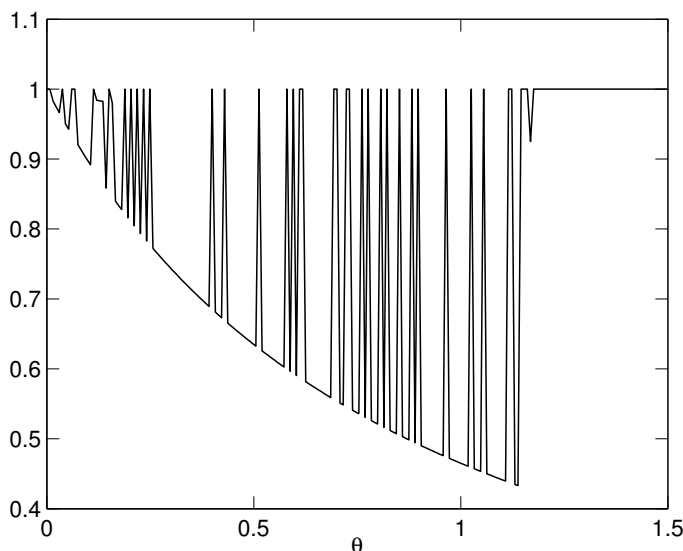


Figure 15.1. Underestimation ratio for Algorithm 15.4 for 5×5 matrix $A(\theta)$ of (15.6) with 200 equally spaced values of $\theta \in [0, 1.5]$.

flavour. Hence if the algorithm is applied repeatedly to a given matrix it may not give the same estimate each time. The randomness lessens the importance of counterexamples, since any counterexample will be valid only for particular starting matrices.

- Working with $t > 1$ columns in the iteration matrix can lead to a vector from the current iteration being parallel to one from the current iteration or an earlier iteration, which results in redundant computation. This redundancy is largely avoided by detecting parallel vectors within the current iteration and the previous one and replacing them by random vectors.
- As for Algorithm 15.4, the algorithm is applicable to both real and complex matrices.

Details of the block algorithm can be found in [624, 2000], and serial and parallel Fortran implementations are described by Cheng and Higham [229, 2001], [230, 2001]. The algorithm is available in MATLAB as function `normest1`, which is invoked by the condition estimation function `condest` (the default is $t = 2$).

15.5. Other Condition Estimators

The first condition estimator to be widely used is the one employed in LINPACK. It was developed by Cline, Moler, Stewart, and Wilkinson [243, 1979]. The idea behind this condition estimator originates with Gragg and Stewart [516, 1976], who were interested in computing an approximate null vector rather than estimating the condition number itself.

We will describe the algorithm as it applies to a triangular matrix $T \in \mathbb{R}^{n \times n}$. There are three steps:

1. Choose a vector d such that $\|y\|$ is as large as possible relative to $\|d\|$, where $T^T y = d$.
2. Solve $Tx = y$.
3. Estimate $\|T^{-1}\| \approx \|x\|/\|y\|$ ($\leq \|T^{-1}\|$).

In LINPACK the norm is the 1-norm, but the algorithm can also be used for the 2-norm or the ∞ -norm. The motivation for step 2 is based on a singular value decomposition analysis. Roughly, if $\|y\|/\|d\|$ ($\approx \|T^{-T}\|$) is large then $\|x\|/\|y\|$ ($\approx \|T^{-1}\|$) will almost certainly be at least as large, and it could be a much better estimate. Notice that $T^T T x = d$, so the algorithm is related to the power method on the matrix $(T^T T)^{-1}$ with the specially chosen starting vector d .

To examine step 1 more closely, suppose that $T = U^T$ is lower triangular and note that the equation $Uy = d$ can be solved by the following column-oriented (saxpy) form of substitution:

```

p(1:n) = 0
for j = n:-1:1
    y_j = (d_j - p_j)/u_jj
(*)   p(1:j-1) = p(1:j-1) + U(1:j-1,j)y(j)
end

```

The idea is to choose the elements of the right-hand side vector d adaptively as the solution proceeds, with $d_j = \pm 1$. At the j th stage of the algorithm $d_n, \dots, d_{j+1}$ have been chosen and $y_n, \dots, y_{j+1}$ are known. The next element $d_j \in \{+1, -1\}$ is chosen so as to maximize a weighted sum of $d_j - p_j$ and the partial sums $p_1, \dots, p_j$, which would be computed during the next execution of statement (*) above. Hence the algorithm looks ahead, trying to gauge the effect of the choice of d_j on future solution components. This heuristic algorithm for choosing d is expressed in detail as follows.

Algorithm 15.5 (LINPACK condition estimator). Given a nonsingular upper triangular matrix $U \in \mathbb{R}^{n \times n}$ and a set of nonnegative weights $\{w_i\}$, this algorithm computes a vector y such that $Uy = d$, where the elements $d_j = \pm 1$ are chosen to make $\|y\|$ large.

```

p(1:n) = 0
for j = n:-1:1
    y_j^+ = (1 - p_j)/u_jj
    y_j^- = (-1 - p_j)/u_jj
    p^+(1:j-1) = p(1:j-1) + U(1:j-1,j)y_j^+(j)
    p^-(1:j-1) = p(1:j-1) + U(1:j-1,j)y_j^-(j)
    if w_j|1 - p_j| + \sum_{i=1}^{j-1} w_i|p_i^+| \geq w_j|1 + p_j| + \sum_{i=1}^{j-1} w_i|p_i^-|
        y_j = y_j^+
    else
        y_j = y_j^-
    end
    p(1:j-1) = p^+(1:j-1)
end

```

```

else
     $y_j = y_j^-$ 
     $p(1:j-1) = p^-(1:j-1)$ 
end
end

```

Cost: $4n^2$ flops.

LINPACK takes the weights $w_j \equiv 1$, though another possible (but more expensive) choice would be $w_j = 1/|u_{jj}|$, which corresponds to how p_j is weighted in the expression $y_j = (d_j - p_j)/u_{jj}$.

To estimate $\|A^{-1}\|_1$ for a full matrix A , the LINPACK estimator makes use of an LU factorization of A . Given $PA = LU$, the equations solved are $U^T z = d$, $L^T y = z$, and $Ax = P^T y$, where for the first system d is constructed by the analogue of Algorithm 15.5 for lower triangular matrices; the estimate is $\|x\|_1/\|y\|_1 \approx \|A^{-1}\|_1$. Since d is chosen without reference to L , there is an underlying assumption that any ill condition in A is reflected in U . This assumption may not be true; see Problem 15.4.

In contrast to the LAPACK norm estimator, the LINPACK estimator requires explicit access to the elements of the matrix. Hence the estimator cannot be used to estimate componentwise condition numbers. Furthermore, separate code has to be written for each different type of matrix and factorization. Consequently, while LAPACK has just a single norm estimation routine, which is called by many other routines, LINPACK has multiple versions of its algorithm, each tailored to the specific matrix or factorization.

Several years after the LINPACK condition estimator was developed, several parametrized counterexamples were found by Cline and Rew [244, 1983]. Numerical counterexamples can also be constructed by direct search, as shown in §26.3.1. Despite the existence of these counterexamples the LINPACK estimator has been widely used and is regarded as being almost certain to produce an estimate correct to within a factor 10 in practice.

A 2-norm condition estimator was developed by Cline, Conn, and Van Loan [245, 1982, Algorithm 1]; see also Van Loan [1181, 1987] for another explanation. The algorithm builds on the ideas underlying the LINPACK estimator by using “look-behind” as well as look-ahead. It estimates $\sigma_{\min}(R) = \|R^{-1}\|_2^{-1}$ or $\sigma_{\max}(R) = \|R\|_2$ for a triangular matrix R , where $\sigma_{\min}$ and $\sigma_{\max}$ denote the smallest and largest singular values, respectively. Full matrices can be treated if a factorization $A = QR$ is available (Q orthogonal, R upper triangular), since R and A have the same singular values. The estimator performs extremely well in numerical tests, often producing an estimate that has some correct digits [245, 1982], [582, 1987]. No counterexamples to the estimator were known until Bischof [115, 1990] obtained counterexamples as a by-product of the analysis of a different but related method, mentioned at the end of this section.

We now describe some methods with intrinsic randomness, for which the output of the method depends on the particular random numbers chosen. A natural idea along these lines is to apply the power method to the matrix $(AA^T)^{-1}$ with a randomly chosen starting vector. If a factorization of A is available, the power method vectors can be computed inexpensively by solving linear systems with A and A^T . Analysis based on the singular value decomposition suggests that there

is a high probability that a good estimate of $\|A^{-1}\|_2$ will be obtained. This notion is made precise by Dixon [340, 1983], who proves the following result.

Theorem 15.6 (Dixon). *Let $A \in \mathbb{R}^{n \times n}$ be nonsingular and let $\theta > 1$ be a constant. If $x \in \mathbb{R}^n$ is a random vector from the uniform distribution on the unit sphere $S_n = \{y \in \mathbb{R}^n : y^T y = 1\}$, then the inequality*

$$(x^T (AA^T)^{-k} x)^{1/2k} \leq \|A^{-1}\|_2 \leq \theta (x^T (AA^T)^{-k} x)^{1/2k} \quad (15.7)$$

holds with probability at least $1 - 0.8\theta^{-k/2}n^{1/2}$ ($k \geq 1$). $\square$

Note that the left-hand inequality in (15.7) always holds; it is only the right-hand inequality that is in question.

For $k = 1$, (15.7) can be written as

$$\|A^{-1}x\|_2 \leq \|A^{-1}\|_2 \leq \theta \|A^{-1}x\|_2,$$

which suggests the simple estimate $\|A^{-1}\|_2 \approx \|A^{-1}x\|_2$, where x is chosen randomly from the uniform distribution on S_n . Such vectors x can be generated from the formula

$$x_i = z_i / \|z\|_2,$$

where $z_1, \dots, z_n$ are independent random variables from the normal $N(0, 1)$ distribution [744, 1998, §3.4.1, E(6)]. If, for example, $n = 100$ and θ has the rather large value 6400 then inequality (15.7) holds with probability at least 0.9.

In order to take a smaller constant θ , for fixed n and a desired probability, we can use larger values of k . If $k = 2j$ is even then we can simplify (15.7), obtaining

$$\|(AA^T)^{-j}x\|_2^{1/2j} \leq \|A^{-1}\|_2 \leq \theta \|(AA^T)^{-j}x\|_2^{1/2j}, \quad (15.8)$$

and the minimum probability stated by the theorem is $1 - 0.8\theta^{-j}n^{1/2}$. Taking $j = 3$, for the same value $n = 100$ as before, we find that (15.8) holds with probability at least 0.9 for the considerably smaller value $\theta = 4.31$.

Probabilistic condition estimation has not yet been adopted in any major software packages, perhaps because the other techniques work so well. For more on the probabilistic power method approach see Dixon [340, 1983], Higham [582, 1987], and Kuczyński and Woźniakowski [753, 1992] (who also analyse the more powerful Lanczos method with a random starting vector). For a probabilistic condition estimation method of very general applicability see Kenney and Laub [728, 1994] and Gudmundsson, Kenney, and Laub [529, 1995], and for applications see Kenney, Laub, and Reese [726, 1998], [727, 1998].

The condition estimators described above assume that a single estimate is required for a matrix given in its entirety. Condition estimators have also been developed for more specialized situations. Bischof [115, 1990] develops a method for estimating the smallest singular value of a triangular matrix that processes the matrix a row or a column at a time. This “incremental condition estimation” method can be used to monitor the condition of a triangular matrix as it is generated, and so is useful in the context of matrix factorizations such as the QR factorization with column pivoting. The estimator is generalized to sparse

matrices by Bischof, Lewis, and Pierce [116, 1990]. Barlow and Vemulapati [76, 1992] develop a 1-norm incremental condition estimator with look-ahead for sparse matrices.

Condition estimates are also required in applications where a matrix factorization is repeatedly updated as a matrix undergoes low rank changes. Algorithms designed for a recursive least squares problem and employing the Lanczos method are described by Ferng, Golub, and Plemmons [411, 1991]. Pierce and Plemmons [941, 1992] describe an algorithm for use with the Cholesky factorization as the factorization is updated, while Shroff and Bischof [1039, 1992] treat the QR factorization.

15.6. Condition Numbers of Tridiagonal Matrices

For a bidiagonal matrix B , $|B^{-1}| = M(B)^{-1}$ (see §8.3), so the condition numbers $\kappa_{E,f}$ and $\text{cond}_{E,f}$ can be computed exactly with an order of magnitude less work than is required to compute B^{-1} explicitly. This property holds more generally for several types of tridiagonal matrix, as a consequence of the following result. Recall that the LU factors of a tridiagonal matrix are bidiagonal and may be computed using the formulae (9.19).

Theorem 15.7. *If the nonsingular tridiagonal matrix $A \in \mathbb{R}^{n \times n}$ has the LU factorization $A = LU$ and $|L||U| = |A|$, then $|U^{-1}||L^{-1}| = |A^{-1}|$.*

Proof. Using the notation of (9.18), $|L||U| = |A| = |LU|$ if and only if, for all i ,

$$|l_i e_{i-1} + u_i| = |l_i| |e_{i-1}| + |u_i|,$$

that is, if

$$\text{sign} \left(\frac{l_i e_{i-1}}{u_i} \right) = 1. \quad (15.9)$$

Using the formulae

$$\begin{aligned} (U^{-1})_{ij} &= \frac{1}{u_j} \prod_{p=i}^{j-1} \left(\frac{-e_p}{u_p} \right), \quad j \geq i, \\ (L^{-1})_{ij} &= \prod_{p=j}^{i-1} (-l_{p+1}), \quad i \geq j, \end{aligned} \quad (15.10)$$

we have

$$\begin{aligned} (U^{-1}L^{-1})_{ij} &= \sum_{k=\max(i,j)}^n (U^{-1})_{ik} (L^{-1})_{kj} \\ &= \sum_{k=\max(i,j)}^n \frac{1}{u_k} \prod_{p=i}^{k-1} \left(\frac{-e_p}{u_p} \right) \prod_{p=j}^{k-1} (-l_{p+1}) \\ &= \prod_{p=i}^{\max(i,j)-1} \left(\frac{-e_p}{u_p} \right) \cdot \prod_{p=j}^{\max(i,j)-1} (-l_{p+1}) \end{aligned}$$

$$\begin{aligned}
& \times \sum_{k=\max(i,j)}^n \frac{1}{u_k} \prod_{p=\max(i,j)}^{k-1} \left(\frac{e_p l_{p+1}}{u_p} \right) \\
& = \prod_{p=i}^{\max(i,j)-1} \left(\frac{-e_p}{u_p} \right) \cdot \prod_{p=j}^{\max(i,j)-1} (-l_{p+1}) \\
& \quad \times \frac{1}{u_{\max(i,j)}} \cdot \sum_{k=\max(i,j)}^n \prod_{p=\max(i,j)}^{k-1} \left(\frac{e_p l_{p+1}}{u_{p+1}} \right).
\end{aligned}$$

Thus, in view of (15.9), it is clear that $|U^{-1}L^{-1}|_{ij} = (|U^{-1}||L^{-1}|)_{ij}$, as required. $\square$

Since L and U are bidiagonal, $|U^{-1}| = M(U)^{-1}$ and $|L^{-1}| = M(L)^{-1}$. Hence, if $|A| = |L||U|$, then, from Theorem 15.7,

$$|A^{-1}|y = |U^{-1}||L^{-1}|y = M(U)^{-1}M(L)^{-1}y. \quad (15.11)$$

It follows that we can compute any of the condition numbers or forward error bounds of interest *exactly* by solving two bidiagonal systems. The cost is $O(n)$ flops, as opposed to the $O(n^2)$ flops needed to compute the inverse of a tridiagonal matrix.

When does the condition $|A| = |L||U|$ hold? Theorem 9.12 shows that it holds if the tridiagonal matrix A is symmetric positive definite, totally positive, or an M -matrix. So for these types of matrix we have a very satisfactory way to compute the condition number.

If A is tridiagonal and diagonally dominant by rows, then we can compute in $O(n)$ flops an upper bound for the condition number that is not more than a factor $2n - 1$ too large.

Theorem 15.8. *Suppose the nonsingular, row diagonally dominant tridiagonal matrix $A \in \mathbb{R}^{n \times n}$ has the LU factorization $A = LU$. Then, if $y \geq 0$,*

$$\| |U^{-1}||L^{-1}|y \|_{\infty} \leq (2n - 1) \| |A^{-1}|y \|_{\infty}.$$

Proof. We have $L^{-1} = UA^{-1}$, so

$$|U^{-1}||L^{-1}|y \leq |U^{-1}||U||A^{-1}|y,$$

and the result follows on taking norms and applying Lemma 8.8. $\square$

In fact, it is possible to compute $|A^{-1}|y$ exactly in $O(n)$ operations for *any* tridiagonal matrix. This is a consequence of the special form of the inverse of a tridiagonal matrix.

Theorem 15.9 (Ikebe). *Let $A \in \mathbb{R}^{n \times n}$ be tridiagonal and irreducible (that is, $a_{i+1,i}$ and $a_{i,i+1}$ are nonzero for all i). Then there are vectors x , y , p , and q such that*

$$(A^{-1})_{ij} = \begin{cases} x_i y_j, & i \leq j, \\ p_i q_j, & i \geq j. \end{cases} \quad \square$$

This result says that the inverse of an irreducible tridiagonal matrix is the upper triangular part of a rank-1 matrix joined along the diagonal to the lower triangular part of another rank-1 matrix. If A is reducible then it has the block form $\begin{bmatrix} A_{11} & 0 \\ A_{21} & A_{22} \end{bmatrix}$ (or its transpose), and this blocking can be applied recursively until the diagonal blocks are all irreducible, at which point the theorem can be applied to the diagonal blocks.

The vectors x , y , p , and q in Theorem 15.9 can all be computed in $O(n)$ flops, and this enables the condition numbers and forward error bounds to be computed also in $O(n)$ flops (see Problem 15.6). Unfortunately, the vectors x , y , p , and q can have a huge dynamic range, causing the computation to break down because of overflow or underflow. For example, for the diagonally dominant tridiagonal matrix with $a_{ii} \equiv 4$, $a_{i+1,i} = a_{i,i+1} \equiv 1$, we have $(x_1 = 1) |x_n| \approx \theta^{n-1}$, $|y_1| \approx \theta^{-1}$, and $|y_n| \approx \theta^{-n}$, where $\theta = 2 + \sqrt{3} \approx 3.73$. An ingenious way of exploiting triangular factorizations to compute $\kappa_1(A)$ without explicitly computing the vectors x , y , p , and q , and techniques for avoiding overflow and underflow in these computations, are developed by Dhillon [338, 1998].

15.7. Notes and References

The clever trick (15.1) for converting the norm $\| |A^{-1}|d \|_\infty$ into the norm of a matrix with which products are easily formed is due to Arioli, Demmel, and Duff [30, 1989].

The p -norm power method was first derived and analysed by Boyd [155, 1974] and was later investigated by Tao [1127, 1984]. Tao applies the method to an arbitrary mixed subordinate norm $\|A\|_{\alpha,\beta}$ (see (6.6)), while Boyd takes the α - and β -norms to be p -norms (possibly different). Algorithm 15.1 can be converted to estimate $\|A\|_{\alpha,\beta}$ by making straightforward modifications to the norm-dependent terms. An algorithm that estimates $\|A\|_p$ using the power method with a specially chosen starting vector is developed by Higham [598, 1992]; the method for obtaining the starting vector is outlined in Problem 15.2. The estimate produced by this algorithm is always within a factor $n^{1-1/p}$ of $\|A\|_p$ and the cost is about $70n^2$ flops. A MATLAB M-file `pnorm` implementing this method is part of the Matrix Computation Toolbox (see Appendix D).

The finite convergence of the power method for $p = 1$ and $p = \infty$ holds more generally: if the power method is applied to the norm $\|\cdot\|_{\alpha,\beta}$ and one of the α - and β -norms is polyhedral (that is, its unit ball has a finite number of extreme points), then the iteration converges in a finite number of steps. Moreover, under a reasonable assumption, this number of steps can be bounded in terms of the number of extreme points of the unit balls in the α -norm and the dual of the β -norm. See Bartels [85, 1991] and Tao [1127, 1984] for further details.

Hager [536, 1984] gave a derivation of the 1-norm estimator based on subgradients and used the method to estimate $\kappa_1(A)$. That the method is of wide applicability because it accesses A only through matrix-vector products was recognized by Higham, who developed Algorithm 15.4 and its complex analogue and wrote Fortran 77 implementations, which use a reverse communication interface [585, 1988], [591, 1990]. These codes are used in LAPACK, the NAG library, and various other program libraries. Algorithm 15.4 is implemented in ROM on the Hewlett-

Packard HP 48G and HP 48GX calculators (along with several other LAPACK routines), in a form that estimates $\kappa_1(A)$. The Hewlett-Packard implementation is instructive because it shows that condition estimation can be efficient even for small dimensions: on a standard HP 48G, inverting A and estimating its condition number (without being given a factorization of A in either case) both take about 5 seconds for $n = 10$, while for $n = 20$ inversion takes 30 seconds and condition estimation only 20 seconds.

Moler [863, 1978] describes an early version of the LINPACK condition estimator and raises the question of the existence of counterexamples. An early version without look-ahead was incorporated in the Fortran code `decomp` in the book of Forsythe, Malcolm, and Moler [430, 1977].

Matrices for which condition estimators perform poorly can be very hard to find theoretically or with random testing, but for all the estimators described in this chapter they can be found quite easily by applying direct search optimization to the under- or overestimation ratio; see §26.3.1.

Both LINPACK and LAPACK return estimates of the *reciprocal* of the condition number, in a variable `rcond` ≤ 1 . Overflow for a very ill conditioned matrix is thereby avoided, and `rcond` is simply set to zero when singularity is detected.

A simple modification to the LINPACK estimator that can produce a larger estimate is suggested by O’Leary [900, 1980]. For sparse matrices, Grimes and Lewis [526, 1981] suggest a way to reduce the cost of the scaling strategy used in LINPACK to avoid overflow in the condition estimation. Zlatev, Wasniewski, and Schaumburg [1287, 1986] describe their experience in implementing the LINPACK condition estimation algorithm in a software package for sparse matrices.

Stewart [1070, 1980] describes an efficient way to generate random matrices of a given condition number and singular value distribution (see §28.3) and tests the LINPACK estimator on such random matrices.

Condition estimators specialized to the (generalized) Sylvester equation have been developed by Byers [191, 1984], Kågström and Westin [685, 1989], and Kågström and Poromaa [682, 1992].

A survey of condition estimators up to 1987, which includes counterexamples and the results of extensive numerical tests, is given by Higham [582, 1987].

Theorems 15.7 and 15.8 are from Higham [589, 1990]. That $\|A^{-1}\|_\infty$ can be computed in $O(n)$ flops for symmetric positive definite tridiagonal A was first shown in Higham [579, 1986].

Theorem 15.9 has a long history, having been discovered independently in various forms by different authors. The earliest reference we know for the result as stated is Ikebe [658, 1979], where a more general result for Hessenberg matrices is proved. A version of Theorem 15.9 for symmetric tridiagonal matrices was proved by Bukhberger and Emel’yanenko [174, 1973]. The culmination of the many papers on inverses of tridiagonal and Hessenberg matrices is a result of Cao and Stewart on the form of the inverse of a block matrix (A_{ij}) with $A_{ij} = 0$ for $i > j + s$ [202, 1986]; despite the generality of this result, the proof is short and elegant. *Any* banded matrix has an inverse with a special “low rank” structure; the earliest reference on the inverse of a general band matrix is Asplund [40, 1959]. For a recent survey on the inverses of symmetric tridiagonal and block tridiagonal matrices see Meurant [845, 1992].

For symmetric positive definite tridiagonal A the standard way to solve $Ax = b$ is by using a Cholesky or LDL^T factorization, rather than an LU factorization. The LINPACK routine SPTSL uses a “twisted” factorization [927, 2000] resulting from the BABE (“burn at both ends”) algorithm, which eliminates from the middle of the matrix to the top and bottom simultaneously (see [341, 1979, Chap. 7] and Higham [579, 1986]). The results of §15.6 are applicable to all these factorizations, with minor modifications.

15.7.1. LAPACK

Algorithm 15.4 is implemented in routine xLACON, which has a reverse communication interface. The LAPACK routines xPTCON and xPTRFS for symmetric positive definite tridiagonal matrices compute condition numbers using (15.11); the LAPACK routines xGTCN and xGTRFS for general tridiagonal matrices use Algorithm 15.4. LINPACK’s tridiagonal matrix routines do not incorporate condition estimation.

Problems

15.1. Show how to rewrite and approximate $\| |A| |A^{-1}| |A| \|_\infty$ so that it can be estimated using the LAPACK norm estimator (Algorithm 15.4).

15.2. (Higham [598, 1992]) The purpose of this problem is to develop a non-iterative method for choosing a starting vector for Algorithm 15.1. The idea is to choose the components of x in the order $x_1, x_2, \dots, x_n$ in an attempt to maximize $\|Ax\|_p / \|x\|_p$. Suppose $x_1, \dots, x_{k-1}$ satisfying $\|x(1:k-1)\|_p = 1$ have been determined and let $\gamma_{k-1} = \|A(:, 1:k-1)x(1:k-1)\|_p$. We now try to choose x_k , and at the same time revise $x(1:k-1)$, to give the next partial product a larger norm. Defining

$$g(\lambda, \mu) = \|\lambda A(:, 1:k-1)x(1:k-1) + \mu A(:, k)\|_p$$

we set

$$x_k = \mu^*, \quad x(1:k-1) \leftarrow \lambda^* x(1:k-1),$$

where

$$g(\lambda^*, \mu^*) = \max \left\{ g(\lambda, \mu) : \left\| \begin{bmatrix} \lambda \\ \mu \end{bmatrix} \right\|_p = 1 \right\}.$$

Then $\|x(1:k)\|_p = 1$ and

$$\gamma_k = \|A(:, 1:k)x(1:k)\|_p \geq \gamma_{k-1}.$$

Develop this outline into a practical algorithm. What can you prove about the quality of the estimate $\|Ax\|_p / \|x\|_p$ that it produces?

15.3. (Higham [591, 1990]) Let the $n \times n$ symmetric tridiagonal matrix $T_n(\alpha) = (t_{ij})$ be defined by

$$t_{ii} = \begin{cases} 2, & i = 1, \\ i, & 2 \leq i \leq n-1, \\ -t_{n,n-1} + \alpha, & i = n, \end{cases}$$

$$t_{i,i+1} = \begin{cases} -((i+1)/2 - \alpha) & \text{if } i \text{ is odd,} \\ -i/2 & \text{if } i \text{ is even.} \end{cases}$$

For example, $T_6(\alpha)$ is given by

$$\begin{bmatrix} 2 & -(1-\alpha) & & & & \\ -(1-\alpha) & 2 & -1 & & & \\ & -1 & 3 & -(2-\alpha) & & \\ & & -(2-\alpha) & 4 & -2 & \\ & & & -2 & 5 & -(3-\alpha) \\ & & & & -(3-\alpha) & 3 \end{bmatrix}.$$

Note that, for all α , $\|T_n(\alpha)e_{n-1}\|_1 = \|T_n(\alpha)\|_1$. Show that if Algorithm 15.3 is applied to $T_n(\alpha)$ with $0 \leq \alpha < 1$ then $x = e_{i-1}$ on the i th iteration, for $i = 2, \dots, n$, with convergence on the n th iteration. Algorithm 15.4, however, terminates after five iterations with $y^5 = T_n(\alpha)e_4$, and

$$\frac{\|y^5\|_1}{\|T_n(\alpha)\|_1} = \frac{8-\alpha}{2n-2-\alpha} \rightarrow 0 \quad \text{as } n \rightarrow \infty.$$

Show that the extra estimate saves the day, so that Algorithm 15.4 returns a final estimate that is within a factor 3 of the true norm, for any $\alpha < 1$.

15.4. Let $PA = LU$ be an LU factorization with partial pivoting of $A \in \mathbb{R}^{n \times n}$. Show that

$$\frac{\|A^{-1}\|_\infty}{2^{n-1}} \leq \|U^{-1}\|_\infty \leq n\|A^{-1}\|_\infty.$$

15.5. Investigate the behaviour of Algorithms 15.3 and 15.4 for

- (a) (Higham [585, 1988]) the Pei matrix, $A = \alpha I + ee^T$ ($\alpha \geq 0$), and for the upper bidiagonal matrix with 1s on the diagonal and the first superdiagonal, and
- (b) (Dhillon [338, 1998]) the inverse of the tridiagonal matrix with zero diagonal and ones on the sub- and superdiagonals.

15.6. (Ikebe [658, 1979], Higham [579, 1986]) Let $A \in \mathbb{R}^{n \times n}$ be nonsingular, tridiagonal, and irreducible. By equating the last columns in $AA^{-1} = I$ and the first rows in $A^{-1}A = I$, show how to compute the vectors x and y in Theorem 15.9 in $O(n)$ flops. Hence obtain an $O(n)$ flops algorithm for computing $\| |A^{-1}|d \|_\infty$, where $d \geq 0$.

15.7. The representation of Theorem 15.9 for the inverse of nonsingular, tridiagonal, and irreducible $A \in \mathbb{R}^{n \times n}$ involves $4n$ parameters, yet A depends only on $3n-2$ parameters. Obtain an alternative representation that involves only $3n-2$ parameters. (Hint: symmetrize the matrix.)

15.8. (RESEARCH PROBLEM) Let $A \in \mathbb{R}^{n \times n}$ be diagonally dominant by rows, let $A = LU$ be an LU factorization, and let $y \geq 0$. What is the maximum size of $\| |U^{-1}| |L^{-1}| y \|_\infty / \|A^{-1}y\|_\infty$? This is an open problem raised in [589, 1990]. In a small number of numerical experiments with full random matrices the ratio has been found to be less than 2 [589, 1990], [885, 1986].